Alternative Ways to Prompt GitHub Copilot in Visual Studio Code

Learning Objectives

By the end of this lesson, you should be able to:

- Use **inline Copilot Chat** directly inside a source file.
- Accept or discard code generated by inline Copilot.
- Refine prompts when the generated result is not suitable.
- Use Copilot's inline completions while writing ordinary code.
- Run generated JavaScript code from the Visual Studio Code terminal.
- Use **comments as prompts** for GitHub Copilot.
- Understand how file context influences comment-driven suggestions.
- Use Copilot to answer conceptual software-engineering questions directly inside a source file.
- Distinguish between explicit prompting, inline completion, and comment-based interaction.

---

1. Introduction

In the previous lesson, we used the dedicated **GitHub Copilot Chat** interface in Visual Studio Code.

That is not the only way to interact with Copilot.

Copilot can also work much closer to the source code you are currently editing.

In this lesson, we will explore two additional interaction styles:

1. **Inline Copilot Chat**
2. **Comment-based prompting**

These approaches allow developers to interact with Copilot without continuously moving to a separate chat panel.

The overall idea is:

```
Source File
    ↓
Ask Copilot inline
```

```
    ↓
Review generated code
    ↓
Accept / Discard / Refine
```

or:

```
Write a meaningful comment
          ↓
Copilot interprets the comment
          ↓
Copilot suggests text or code
          ↓
Developer accepts or continues typing
```

These approaches can help keep the developer focused on the code currently being written.

---

2. Inline Copilot Chat

Inline Copilot Chat allows you to ask Copilot for assistance directly from the editor.

Instead of opening a separate chat panel, you invoke Copilot while your cursor is inside a source file.

Conceptually:

```
Code Editor
    +
Current Cursor Position
    +
Prompt
    ↓
Inline Copilot
    ↓
Suggested change or generated code
```

This interaction style can be particularly useful when the requested output should be inserted directly into the current file.

NOTE — UI and Shortcuts Can Change

GitHub Copilot and Visual Studio Code are updated frequently. The exact keyboard shortcut, icon, or placement of inline chat controls may vary by version and operating system. Use the shortcut or UI action shown by your current VS Code installation.

---

3. Lab Setup — Create a New JavaScript File

We will begin with a simple JavaScript example.

Step 1 — Create a File

Inside your project folder, create:

`sample2.js`

Open the file in Visual Studio Code.

Because the file extension is .js, Copilot now has an important contextual clue:

Current Language → JavaScript

---

4. Opening Inline Copilot Chat

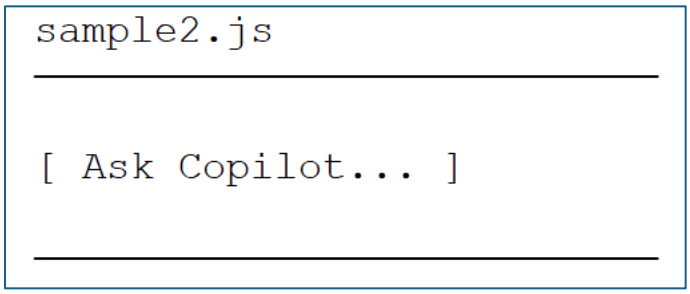
With the cursor inside sample2.js, invoke inline Copilot Chat.

Depending on your environment, this may be available through:

- a keyboard shortcut;
- a Copilot icon;
- an inline action;
- a context menu.

When opened, the inline prompt box appears directly inside or near the editor.

The interaction is approximately:



This differs from the main chat panel because the interaction is closely associated with the currently active source file.

---

5. Ask Copilot to Generate a Function

Suppose we want a JavaScript function that formats a Date object as a string.

We can enter a prompt such as:

Create a JavaScript function that formats a Date object

as a readable date string.

Submit the prompt.

Copilot may generate code directly in the editor.

For example:

```
function formatDate(date) {
    return date.toLocaleDateString();
}
```

The exact generated implementation may differ.

This is normal.

---

6. Reviewing Inline Generated Code

When Copilot proposes an inline change, Visual Studio Code may provide actions such as:

Accept

Discard

The workflow becomes:

```
Prompt
   ↓
Generated code
   ↓
Review
   ↓
Accept or Discard
```

Accept

Selecting **Accept** applies the proposed code to the current file.

Discard

Selecting **Discard** removes the proposed change and returns you to the original file state.

WARNING — Accept Is Not the Same as Approve

Clicking **Accept** only means that you want the code inserted into the editor. It does not prove that the implementation is correct.

Always review the generated code before accepting it.

---

7. Refining an Unsatisfactory Response

If Copilot does not generate what you expected, do not simply keep the first answer.

Instead, refine your prompt.

Suppose the original request was:

Format a date.

This is vague.

A better prompt could be:

Create a JavaScript function named formatDate that accepts a Date object and returns a string in ***`YYYY-MM-DD`*** *format*.

The second version is more specific.

It defines:

- language;
- function name;
- input type;
- output format.

The refinement process is:

```
Initial Prompt
      ↓
Generated Result
      ↓
Not Good Enough
      ↓
Add More Context / Requirements
      ↓
```

```
Submit Again
      ↓
Better-Targeted Result
```

TIP — Refine Rather Than Repeatedly Resubmitting the Same Prompt

If the response is unsuitable, identify what information was missing. Add that information to the next prompt.

---

8. AI Responses May Vary

AI-generated responses are not guaranteed to be identical every time.

Even if the prompt remains similar, Copilot may produce a different:

- function name;
- implementation;
- coding style;
- API choice;
- explanation;
- formatting approach.

For example, a date-formatting task could potentially produce:

```
date.toLocaleDateString();
```

or:

```
date.toISOString().split("T")[0];
```

or a manually constructed string.

All three approaches may behave differently.

PITFALL — Expecting Deterministic Output

Do not assume that repeating the same prompt will always generate the exact same result. Treat each generated response as a new proposal that must be evaluated.

---

9. Inline Code Completion

Copilot can also suggest code without an explicit chat prompt.

This appears as an inline completion while you type.

For example, suppose your file contains:

```
function formatDate(date) {
    return date.toLocaleDateString();
}
```

After this function, you begin typing:

export

Copilot may predict that you intend to export the function and suggest:

```
export { formatDate };
```

You may then accept the suggestion using the appropriate completion key, often Tab.

The workflow is:

```
You begin typing
      ↓
Copilot predicts likely continuation
      ↓
Inline suggestion appears
      ↓
Press Tab to accept
```

or

Continue typing to ignore/replace

---

10. You Are Not Required to Accept Suggestions

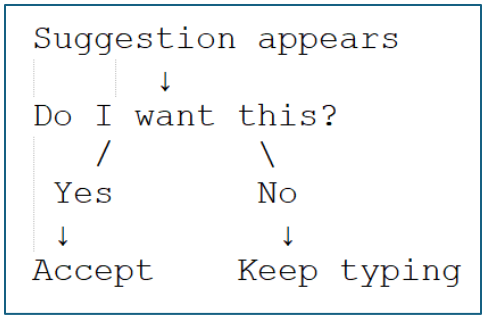
Suppose Copilot suggests:

```
export { formatDate };
```

but exporting the function is not part of your current goal.

You can simply continue typing your own code.

The developer remains in control.



NOTE — Suggestions Are Optional

Copilot completions should be treated as proposals. If they do not match your design, continue writing your own code.

---

11. Generating Code from Inline UI Controls

Visual Studio Code may also display Copilot-related icons or inline actions near empty lines or relevant code positions.

Selecting these controls can open the inline prompt interface.

This provides multiple ways to reach essentially the same capability:

Keyboard Shortcut

or

Inline Copilot Icon
↓
Inline Prompt
↓
Generated Suggestion

If the inline interface becomes distracting, you can continue using the dedicated Copilot Chat window instead.

The choice depends on the task.

Inline Chat is useful when:

- you want code inserted near the cursor;
- the task relates closely to the current file;
- you want a quick edit.

Main Chat is useful when:

- you want a longer discussion;
- you want explanations;
- you are exploring alternatives;
- you do not immediately want code inserted.

---

12. Completing the Date Example

Assume we have a function:

```
function formatDate(date) {
    return date.toLocaleDateString();
}
```

Now we want to call it and print the result.

Begin typing:

console

Copilot may suggest:

```
console.log(formatDate(new Date()));
```

If the suggestion matches your intention, accept it.

Your file may now contain:

```
function formatDate(date) {
    return date.toLocaleDateString();
}

console.log(formatDate(new Date()));
```

---

13. Modifying the Test Date

Instead of using the current date, we may want to test a specific date.

For example:

```
const testDate = new Date(2024, 4, 15);

console.log(formatDate(testDate));
```

Remember that the numeric JavaScript Date constructor uses a zero-based month when using this form:

January → 0
February → 1
March → 2
April → 3
May → 4
...
December → 11

Therefore:

```
new Date(2024, 4, 15)
```

represents May 15, 2024.

PITFALL — JavaScript Month Numbers

When using the numeric Date constructor, JavaScript months are zero-based. This is a common source of off-by-one errors.

---

14. Copilot Can Suggest Comments Too

While typing code, Copilot may suggest comments that describe the generated value or behavior.

For example:

```
const testDate = new Date(2024, 4, 15); // May 15, 2024
```

This demonstrates that Copilot does not only predict executable statements.

It may suggest:

- comments;
- documentation;
- variable names;
- function calls;
- expected values;
- test examples.

---

15. Running the JavaScript File

Generated code should be executed and verified.

Assume Node.js is installed.

Step 1 — Open the Integrated Terminal

From Visual Studio Code, open a new terminal.

The exact UI may vary, but conceptually:

`Terminal`

↓

`New Terminal`

Step 2 — Run the Program

Execute:

node sample2.js

Press Enter.

You should see the formatted date in the terminal.

For example:

5/15/2024

The exact formatting may depend on:

- operating system;
- locale;
- runtime environment;
- formatting API.

---

16. Verify the Function

Do not stop after observing one successful result.

Try several values.

For example:

```
console.log(formatDate(new Date(2024, 0, 1)));

console.log(formatDate(new Date(2024, 6, 10)));

console.log(formatDate(new Date(2024, 11, 31)));
```

Then compare the results with what you expect.

TIP — Always Test Generated Code

A generated function that works for one input may still fail for another. Test normal cases, boundary cases, and unusual inputs where relevant.

---

17. Another Interaction Style — Prompting Through Comments

Copilot can also respond to ordinary source-code comments.

This is a powerful interaction style because comments can communicate developer intent directly inside the code.

The general pattern is:

```
Comment describing intent
        ↓
Copilot reads context
        ↓
Copilot predicts useful continuation
        ↓
Developer accepts or modifies it
```

This approach is closely related to **comment-driven development**, which will be explored further in later lessons.

---

18. Lab Setup — Create Another File

Create:

`sample3.js`

Open it in Visual Studio Code.

We will use comments to interact with Copilot.

---

19. Writing a Question as a Comment

In JavaScript, a single-line comment begins with:

```
//
```

You can therefore write:

```
// Question: What does SOLID stand for in software development?
```

After pressing Enter, Copilot may suggest an answer as another comment.

For example:

```
// Answer:
// S - Single Responsibility Principle
// O - Open/Closed Principle
// L - Liskov Substitution Principle
// I - Interface Segregation Principle
// D - Dependency Inversion Principle
```

You can accept the suggestion if it is correct.

---

20. SOLID as an Example of Contextual Continuation

Once the file contains:

```
// Question: What does SOLID stand for in software development?
```

and the answer, Copilot now has context about the topic.

If you begin another comment such as:

```
// Question: What is the Single
```

Copilot may suggest:

Responsibility Principle?

This happens because the existing file content provides contextual clues.

The file is already discussing:

SOLID

```
    ↓
Single Responsibility
    ↓
Open/Closed
    ↓
Liskov Substitution
    ...
```

Therefore, Copilot can predict likely follow-up questions.

---

21. Asking About the Single Responsibility Principle

You might write:

```
// Question: What is the Single Responsibility Principle?
```

Copilot may suggest an explanation such as:

```
// A class should have only one reason to change.
```

The principle is commonly summarized as:

A software component should have one primary responsibility or reason to change.

This example demonstrates that Copilot can provide **conceptual explanations** through code comments.

---

22. Continuing the Conversation Through File Context

After discussing the Single Responsibility Principle, you may begin writing:

```
// Question: What is the Open
```

Copilot may predict:

```
/Closed Principle?
```

This is not because the phrase is magical.

It is because the current file contains contextual information about SOLID principles.

Conceptually:

```
Existing File Content
        +
Current Comment
        ↓
Copilot
        ↓
Context-Aware Completion
```

---

23. Explicit Question Markers Are Optional

You do not necessarily need to write:

```
// Question:
```

or:

```
// Q:
```

Copilot can often understand an ordinary descriptive comment.

For example:

```
// Explain the Liskov Substitution Principle
```

may be enough.

Or you may simply begin:

```
// Liskov
```

and Copilot may infer the intended continuation from the surrounding SOLID discussion.

NOTE — Keywords Help Humans More Than They Help the Model

Labels such as Question: or Q: can make the file easier for people to read, but Copilot can often infer intent from ordinary comments without those labels.

---

24. Comments as Prompts

A comment can act as a prompt because it communicates intent.

For example:

```
// Create a function that returns true if a number is even
```

Copilot may suggest:

```
function isEven(number) {
    return number % 2 === 0;
}
```

The pattern is:

```
Natural-Language Comment
          ↓
Intent
          ↓
Copilot Suggestion
          ↓
Generated Code
```

This creates a bridge between natural-language design and implementation.

---

25. Comment-Based Prompts Are Not Limited to Questions

Comments can represent different forms of intent.

Explanation Request

```
// Explain why this function uses recursion
```

Implementation Request

```
// Create a function that sorts employees by last name
```

Validation Requirement

```
// Return false if the email address is empty
```

Refactoring Intent

```
// Refactor this code to remove duplicate logic
```

Testing Intent

```
// Create test cases for invalid dates
```

The important idea is that the comment gives Copilot something meaningful to work with.

---

26. Why Comments Can Be Powerful Context

Consider this code:

```
function process(data) {
```

This gives Copilot relatively little information.

Now compare:

```
// Filter inactive employees, sort active employees by last name,
// and return only their name and email address.
function process(data) {
```

The second version contains far more intent.

Copilot can infer:

```
Input       → employee data
Filter      → active employees
Sort        → last name
Output      → name + email
```

This often leads to a more targeted suggestion.

TIP — Write Comments That Express Intent

Comments that explain *what the code should accomplish* are usually more useful than comments that merely restate syntax.

---

27. Poor Comment vs. Useful Comment

Consider:

```
// loop
```

This says almost nothing.

Compare:

```
// Iterate through all employees and print the name
```

```
// of each employee whose active property is true.
```

The second comment communicates:

- data source;
- iteration requirement;
- filtering condition;
- expected action.

This is much stronger context.

---

28. Practical Exercise — Comment-Driven Questioning

Create sample3.js and enter:

```
// Question: What does SOLID stand for in software development?
```

Observe Copilot's suggested response.

Accept it only if it is correct.

Then add:

```
// Question: Explain the Single Responsibility Principle.
```

Continue with:

```
// Question: Explain the Open/Closed Principle.
```

Then:

```
// Question: Explain the Liskov Substitution Principle.
```

Observe whether Copilot uses the earlier contents of the file to predict later questions and answers.

---

29. Practical Exercise — Comments to Code

In a new JavaScript file, write:

```
// Create a function that accepts an array of numbers
// and returns only the even numbers.
```

Observe the generated suggestion.

You may receive something similar to:

```
function getEvenNumbers(numbers) {
    return numbers.filter(number => number % 2 === 0);
}
```

Now test it:

```
console.log(getEvenNumbers([1, 2, 3, 4, 5, 6]));
```

Expected result:

```
[2, 4, 6]
```

Then refine the comment:

```
// Create a function that accepts an array of numbers,
// returns only the even numbers,
// and sorts them from largest to smallest.
```

Observe how the new requirement affects the generated code.

---

30. Three Ways of Working with Copilot

At this stage, we have seen three different interaction patterns.

Method 1 — Dedicated Copilot Chat

```
Open Chat
   ↓
Ask Question
   ↓
Receive Explanation / Code
```

Useful for:

- conversations;
- explanations;
- follow-up questions;
- conceptual exploration.

Method 2 — Inline Copilot Chat

```
Cursor in Source File
        ↓
Invoke Inline Chat
        ↓
Describe Desired Change
        ↓
Preview Code
        ↓
Accept / Discard
```

Useful for:

- targeted edits;
- generating code directly in the active file;
- modifying nearby code.

Method 3 — Comment-Based Prompting

```
Write Comment
     ↓
Copilot Reads Intent
     ↓
Inline Completion
     ↓
Accept / Continue Typing
```

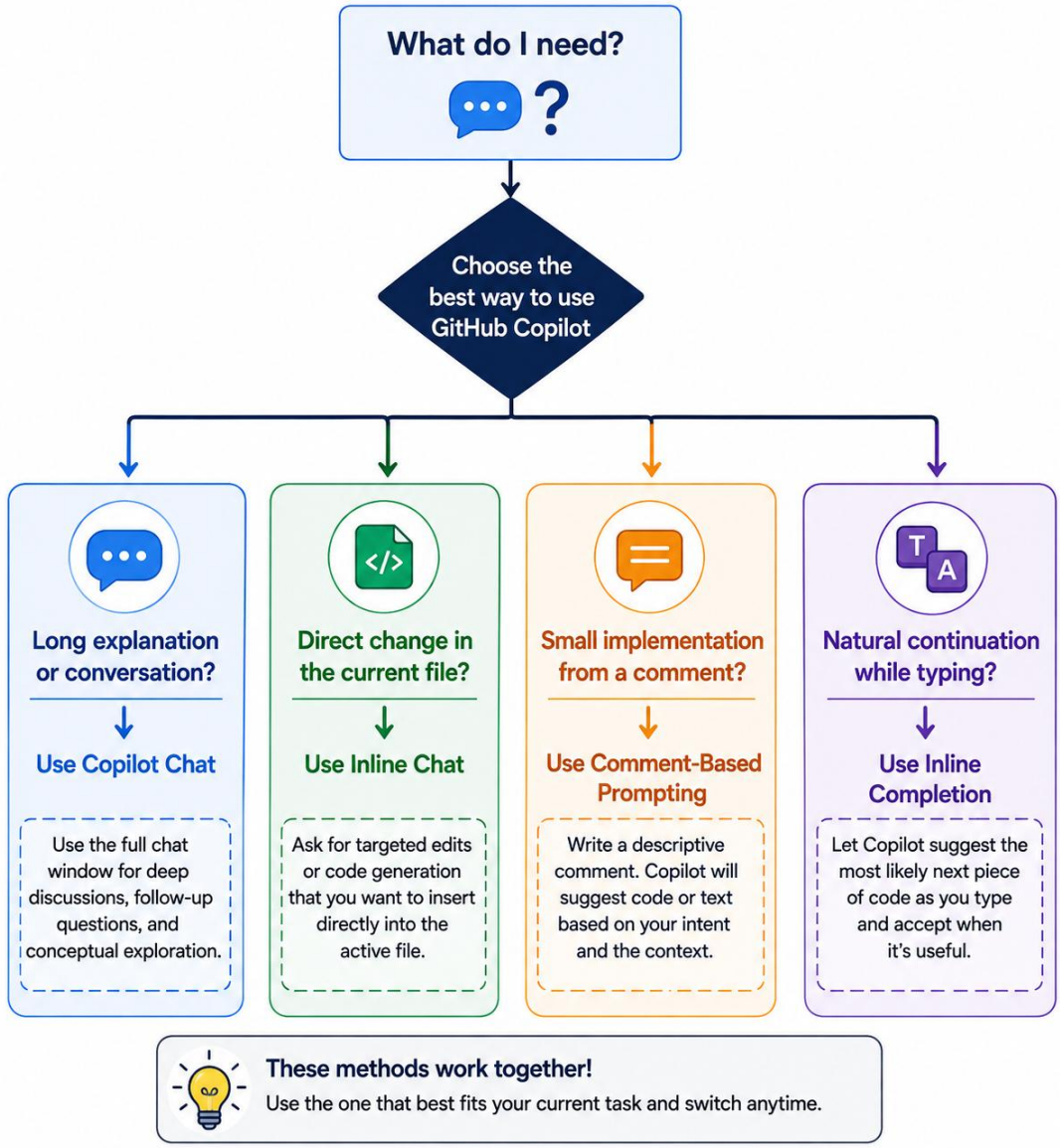
Useful for:

- expressing implementation intent;
- generating code from comments;
- documentation-driven workflows;
- quick conceptual questions.

---

31. Choosing the Appropriate Interaction Style

A useful decision model is:



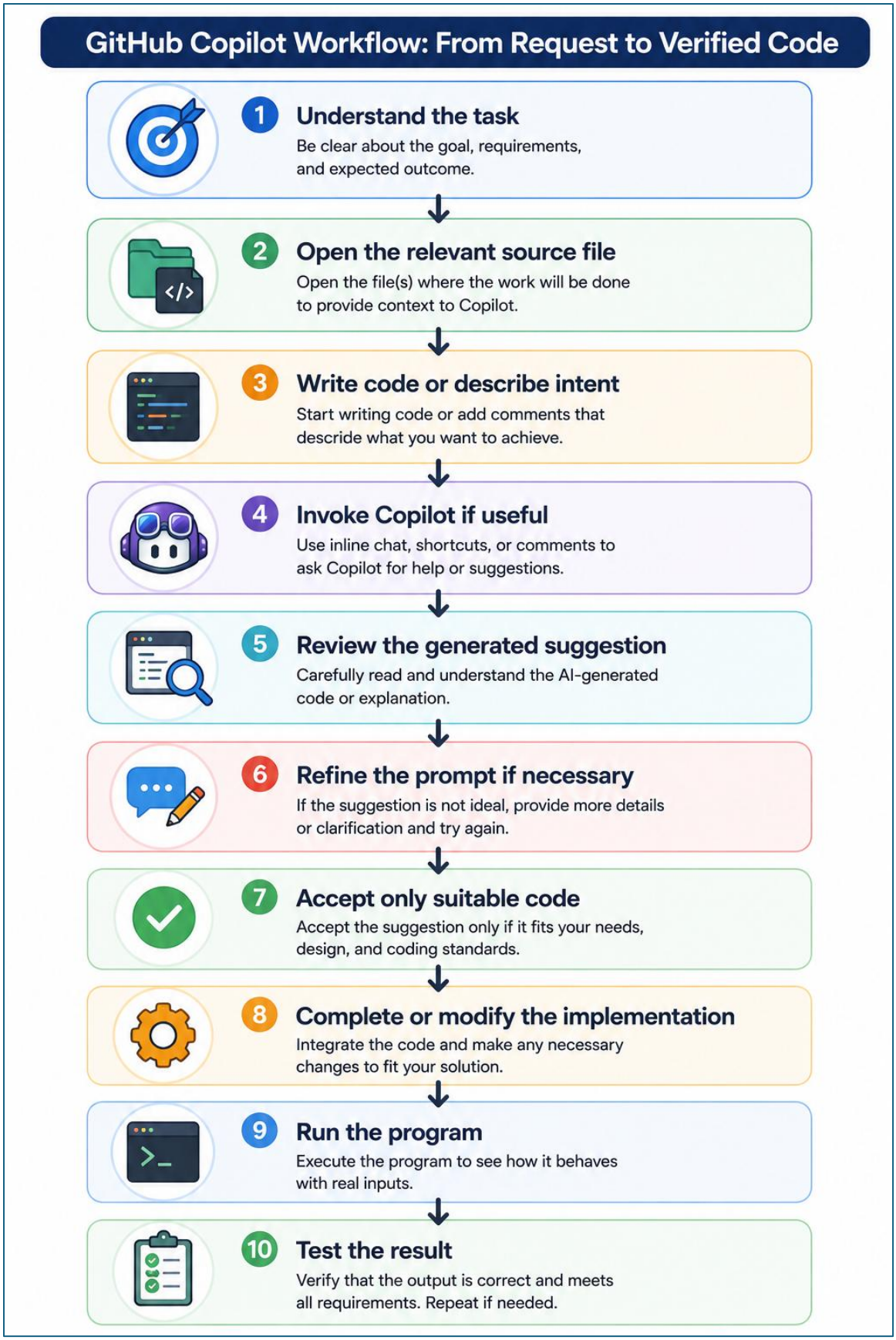
These interaction methods complement each other.

There is no requirement to use only one.

---

32. Recommended Development Workflow

A disciplined workflow for inline Copilot use is:


GitHub Copilot Workflow: From Request to Verified Code
1
Understand the task
Be clear about the goal, requirements, and expected outcome.
2
Open the relevant source file
Open the file(s) where the work will be done to provide context to Copilot.
3
Write code or describe intent
Start writing code or add comments that describe what you want to achieve.
4
Invoke Copilot if useful
Use inline chat, shortcuts, or comments to ask Copilot for help or suggestions.
5
Review the generated suggestion
Carefully read and understand the AI-generated code or explanation.
6
Refine the prompt if necessary
If the suggestion is not ideal, provide more details or clarification and try again.
7
Accept only suitable code
Accept the suggestion only if it fits your needs, design, and coding standards.
8
Complete or modify the implementation
Integrate the code and make any necessary changes to fit your solution.
9
Run the program
Execute the program to see how it behaves with real inputs.
10
Test the result
Verify that the output is correct and meets all requirements. Repeat if needed.

WARNING — Avoid "Generate and Forget" Development

Inline generation makes accepting code extremely easy. This convenience increases the importance of disciplined review. The faster code enters your project, the easier it is to overlook incorrect assumptions.

---

33. Common Pitfalls

Pitfall 1 — Accepting the First Suggestion Automatically

A first response may be acceptable, but it may also be incomplete or inappropriate.

Use:

```
Generate
    ↓
Evaluate
    ↓
Refine if needed
```

rather than:

```
Generate
    ↓
Accept immediately
```

Pitfall 2 — Writing Vague Prompts

For example:

Make a date function.

A stronger request is:

Create a JavaScript function named formatDate that accepts

a Date object and returns the date in YYYY-MM-DD format.

Pitfall 3 — Assuming Completion Means Intent

Copilot may suggest an export statement, comment, or function call simply because it appears likely from the current context.

A likely continuation is not necessarily what your design requires.

Pitfall 4 — Not Running Generated Code

A function that looks correct may behave differently when executed.

Always verify the result.

Pitfall 5 — Treating Comments as Specifications Without Review

Comments can produce impressive code suggestions, but Copilot may interpret them incorrectly.

The developer must still compare implementation against requirements.

---

34. Tips for Effective Inline Copilot Use

TIP — Keep Relevant Code Nearby

Useful surrounding code can provide additional context for Copilot.

TIP — Describe Behavior, Not Just Syntax

Prefer:

Create a function that returns active employees sorted by last name

over:

Write a loop.

TIP — Refine Incrementally

If the first result is close but incomplete, add the missing requirement rather than rewriting everything.

TIP — Use the Right Interaction Mode

Use full Chat for discussion, inline Chat for targeted edits, and comments for concise intent-driven generation.

---

35. Key Concepts

Concept	Description
Inline Copilot Chat	Prompting Copilot directly from the code editor
Inline Completion	Predicted continuation displayed while typing
Accept	Apply the generated suggestion
Discard	Reject the generated suggestion
Prompt Refinement	Improve the instruction based on an unsatisfactory result
Comment-Based Prompting	Use comments to communicate intent to Copilot
Context-Aware Completion	Suggestion influenced by existing file contents
Comment-Driven Development	Using descriptive comments to guide code generation

---

36. Lesson Summary

GitHub Copilot can be used in several ways beyond the dedicated chat window.

In this lesson, we explored:

```
Inline Chat
     +
Inline Completion
     +
Comment-Based Prompting
```

Inline Chat allows a developer to:

```
Prompt directly from the editor
        ↓
Preview generated code
        ↓
Accept / Discard
        ↓
Continue coding
```

Inline completion allows Copilot to predict likely next code as the developer types.

Comment-based prompting allows ordinary comments to communicate intent:

```
// Create a function that returns all active employees
```

which can lead Copilot toward an implementation.

The main principle remains unchanged:

```
Developer Intent
```

```
      +
Relevant Context
      ↓
Copilot Suggestion
      ↓
Developer Review
      ↓
Run and Test
```

Copilot can accelerate implementation, but it should remain part of a controlled software-development process.

FINAL NOTE

The value of inline Copilot is not simply that it can write code faster. Its real value comes from integrating AI assistance directly into the developer's normal workflow while allowing the developer to remain responsible for the design, interpretation, testing, and final implementation.